

SARA: Segment Aware Rate Adaptation Algorithm for Dynamic Adaptive Streaming Over HTTP

Parikshit Juluri*, Venkatesh Tamarapalli[†], and Deep Medhi*

*University of Missouri - Kansas City, [†]Indian Institute of Technology–Guwahati, India

Email: *{pjuluri, dmedhi}@umkc.edu, [†]t.venkat@iitg.ernet.in

Abstract—Dynamic adaptive HTTP (DASH) based streaming is steadily becoming the most popular online video streaming technique. DASH streaming provides seamless playback by adapting the video quality to the network conditions during the video playback. A DASH server supports adaptive streaming by hosting multiple representations of the video and each representation is divided into small segments of equal playback duration. At the client end, the video player uses an adaptive bitrate selection (ABR) algorithm to decide the bitrate to be selected for each segment depending on the current network conditions. Currently, proposed ABR algorithms ignore the fact that the segment sizes significantly vary for a given video bitrate. Due to this, even though an ABR algorithm is able to measure the network bandwidth, it may fail to predict the time to download the next segment. In this paper, we propose a segment-aware rate adaptation (SARA) algorithm that considers the segment size variation in addition to the estimated path bandwidth and the current buffer occupancy to accurately predict the time required to download the next segment. We also developed an open source Python based emulated DASH video player, that was used to compare the performance of SARA and a basic ABR. Our results show that SARA provides a significant gain over the basic algorithm in the video quality delivered, without noticeably impacting the video switching rates.

Index Terms—HTTP video streaming, DASH, Adaptive bitrate selection (ABR) algorithm, QoE

I. INTRODUCTION

The use of HTTP-based streaming video services has been increasing over the past few years and the trend is expected to continue. Use of HTTP for multimedia streaming facilitates the service providers to use traditional stateless web servers for streaming videos. HTTP-based delivery also ensures that the users behind a firewall or Network Address Translation (NAT) devices can also easily access the services. Traditional HTTP-based progressive download techniques select the video quality prior to starting the video download. However, due to the diverse client access platforms and the varying network conditions, it is difficult to predict the appropriate video quality for the current session, which could either result in interruptions or a poor video quality selection.

Dynamic Adaptive Streaming over HTTP (DASH) is a fairly new technique, the first open standard being released by MPEG in 2011 [1]. Its ability to support adaptive streaming while retaining the advantages of progressive download based HTTP streaming has helped it to gain significant popularity over the last couple of years. It is being used by some of the most popular online video service providers e.g., Netflix, YouTube, and Hulu [2]. DASH improves the Quality of

Experience (QoE) of the user by dynamically adjusting the video quality during the playback according to the network conditions, thus ensuring smooth playback at the highest possible bitrate.

A DASH server divides a multimedia video into multiple components of video, audio, and subtitle files. For the video component, different representations are created with each varying in video bitrate, dimensions, format, etc. Similarly, the audio and subtitle files are divided into representations varying in languages. Each representation is split into smaller chunks called *segments* that are of fixed playback duration (typically 2-4 seconds). These individual segments are stored as separate files on the web server, each with a unique URL.

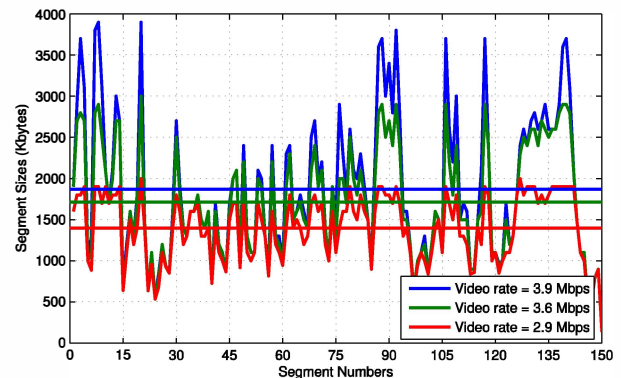


Fig. 1. Variation in Segment Sizes for the Big Buck Bunny Video: 4s segments

For each video, the server creates a Media Presentation Description (MPD) file, which is an XML format file that lists all the representations, segment information, and segment URLs for each representation. At the client side, the DASH player first downloads and parses the MPD file to retrieve the media information. The player can change the representation according to the platform, user preference, and network conditions. A DASH client typically employs an adaptive bitrate selection (ABR) algorithm to pick the most suitable representation. The ABR algorithms proposed in previous literature typically use parameters like the average segment download rate [3], available bandwidth [4], or buffer occupancy [5]. The client typically starts with the lowest representation, and then based on one of the parameters measured during the download, estimates the best representation for each subsequent segment.

Although the segments are of equal playback durations, the

size of these segments could vary significantly [6]. Fig. 1 shows the variation in the segment sizes (all of 4 second durations) for the Big Buck Bunny video [7] for three different representations (3.9 Mbps, 3.6 Mbps, and 2.9 Mbps). For the representation with a 3.9 Mbps rate, we note that the segment size varies from 538 KB to 3.9 MB with the average segment size being 1.8 MB. A similar trend was found in the segment sizes for the other bitrates, as shown in Fig. 1.

The main objective of an ABR algorithm is to predict the amount of time required to download the next segment to ensure it is downloaded before the player consumes the content of the buffer. Most of the ABR algorithms in the literature consider only the path bandwidth and buffer occupancy, but do not consider the variation in the segment sizes in the rate estimation models. Hence, even if an ABR algorithm is able to predict the path bandwidth, it may not be able to predict the download rate of the next segment. We propose a scheme that uses the segment sizes to improve the rate adaptation in [8]. In this paper, we develop a Segment-Aware Rate Adaptation (SARA) algorithm, that not only uses the estimated path bandwidth and current buffer occupancy but also, the segment sizes to predict the time required to download the next segment. This ensures that the best possible representation of the video is downloaded while avoiding video buffer starvation.

The rest of this paper is organized as follows. Section II discusses related work in the literature that has proposed different DASH ABR techniques. In Section III, we present the proposed segment-aware rate adaptation algorithm, SARA. We describe our evaluation setup and present the results showing the performance of SARA compared with the basic ABR algorithm in Section IV. We conclude the paper in Section V.

II. RELATED WORK

The main objective of a DASH system is to design an ABR algorithm that can pick the most suitable video bitrate while avoiding interruptions in the playback. While most of the commercial players implement ABR algorithms, little is known about the exact techniques used. The ABR algorithms used in two commercial DASH video players (Netflix and Microsoft's Smooth Streaming) and in an open source player (Adobe OSMF) were evaluated in [9]. It was found that all three clients had significant shortcomings. They were found to be either too slow to converge to the optimum bitrate, resulting in frequent bitrate switching, or were too slow to start. Hence, they conclude that there is a need to develop ABR algorithms that can overcome these shortcomings.

Recently, some details about Netflix's ABR algorithm were published in [6]. According to this work, it mainly uses buffer occupancy to pick up a suitable representation for the future segments. While the estimation of the path bandwidth is an integral part of any ABR algorithm, the authors observed that due to the network dynamics it is not easy to predict the throughput based on past observations, and hence, they limit the throughput estimation to the initial stage. They used a video buffer reservoir that works as a cushion to account for the variations in the network capacity and segment sizes. Here, they study the variation in the segment sizes of the downloaded

segments to update the reservoir capacity. However, the video segment sizes can vary significantly, and are dependent on several factors such as the encoding scheme used, the number of frame changes within the video, and the content of the video, thus making it difficult to predict the segment sizes based on previous data. In our approach, by including the segment sizes in the enhanced MPD file, the client can obtain the exact segment sizes, and thus, estimate the segment download times with better accuracy.

An ABR algorithm that estimates the bandwidth variations based on the segment fetch times was proposed in [3]. In this algorithm, the authors used smoothed throughput measurements in conjunction with a step-wise increase/aggressive decrease method to switch between the video bitrates. However, the aggressive decrease approach results in frequent fluctuations in the video bitrate that can affect the users' QoE. The algorithm proposed in [10] also used the average download rate as an input. However, they avoided frequent bitrate switching by dividing the playback into two phases: the *Fast Start* phase during which they aggressively increase the bitrate followed by a later stage where they limited the bitrate switching events to provide smooth streaming. The Fair, Efficient, and Stable adaptive (FESTIVE) algorithm [5] used a bandwidth estimator that used a harmonic mean to overcome outliers in the measurements. FESTIVE also used randomized chunk scheduling and delayed switching to overcome synchronizing bias in the network sampling and tune the tradeoff between efficiency and stability, respectively.

In [4] a measurement proxy was used to measure the available network bandwidth, round-trip time (RTT), and the loss rate. Using these measurements, they estimated the highest acceptable video rate. When the preferred bitrate was estimated to be lower than the current bitrate, they proposed using an intermediate level instead of immediately switching to the preferred rate. However, using a measurement proxy can limit the scalability of the system, and also needs additional hardware. Our approach does not need any additional infrastructure and uses a predominantly client-side software-based approach to estimate the segment download times.

Most of the ABR algorithms proposed so far used the download rate and the current buffer occupancy to select the most suitable bitrate for the next segment. They did so by ignoring the variation in the segment size. However, as shown in Fig. 1 the segment sizes vary significantly. Previous research has shown that the data transfer time over HTTP can vary significantly based on the network conditions and the file size distribution [11]. In this paper, we propose an ABR algorithm (SARA) that explicitly considers the segment sizes for the various representations, the network throughput measurement, and buffer occupancy to estimate the time to download the next segment.

III. SEGMENT AWARE RATE ADAPTATION (SARA)

In this section, we provide a detailed description of the proposed SARA algorithm and its associated components: an enhanced MPD file that contains the information about the segment size and a throughput estimation module that considers the segment sizes.

A. Enhanced MPD

A typical MPD file contains information such as: media representation, segment duration, and segment URLs. We propose to enhance the MPD file by listing the sizes of the segments that are later used by the client to make an informed decision in the rate adaptation phase. In our experiments, we added the segment sizes to the standard MPD file during the pre-processing phase at the DASH server. These sizes are later used in the throughput estimation (Section III-B) and SARA algorithm (Section III-C).

B. Throughput Estimation with Weighted Harmonic Mean

In order for the rate adaptation algorithm to pick the appropriate bitrate, it is necessary to measure the throughput for the entire video session consisting of a sequence of downloaded segments. In order to avoid the effect of instantaneous variation in the measured throughput, harmonic mean was used in [5]. However, the segments being downloaded are of different sizes, and the average download rate over HTTP depends on the file size distribution [11]. In order to accurately predict the download rate for the next segment, we assigned weights ($w_1, w_2, \dots$) that were proportional to the segment sizes. These weights and the respective segment download rates ($d_1, d_2, \dots$) were used to calculate the weighted harmonic mean download rate. The weighted harmonic mean download rate for n downloaded segments is given by

$$H_n = \frac{\sum_{i=1}^n w_i}{\sum_{i=1}^n \frac{w_i}{d_i}} \quad (1)$$

Based on the H_n computed for n segments, the time to download the next segment is predicted by w_{n+1}/H_n .

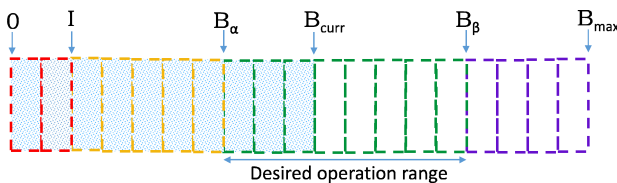


Fig. 2. Adaptive Streaming Buffer

C. Segment Aware Rate Adaptation Algorithm (SARA)

SARA selects the most suitable representation for the next segment to be downloaded from the set of available representations ($\mathbb{R}$). Every segment that is downloaded is placed in a buffer of maximum size B_{max} . The buffer is associated with three thresholds: I , B_α , and B_β as illustrated in Fig. 2. These thresholds are defined in terms of the number of segments. Based on the buffer occupancy at any given time, B_{curr} , the rate adaptation is done in four stages listed as follows:

- **Fast Start** ($B_{curr} \leq I$): When the buffer occupancy is below I , the lowest bitrate is selected. This ensures that the playback start time is kept as small as possible. It was observed that minimizing playback start time is important to prevent the user from abandoning the video session [12].

Algorithm 1: Segment Aware Rate Adaptation Algorithm

Data:

$\mathbb{R}$: Set of available bitrates $\{r^{min}, \dots, r^i, \dots, r^{max}\}$

$I, B_\alpha, B_\beta, B_{max}$: Buffer constants (number of segments)

Input:

n : Segment number of the most recent download

r^{curr} : Bitrate of the most recently downloaded segment

B_{curr} : Current buffer occupancy in seconds

$W_{n+1} = \{w_{n+1}^{min}, \dots, w_{n+1}^i, \dots, w_{n+1}^{max}\}$ The sizes of the segments for bitrates $\{r^{min}, \dots, r^i, \dots, r^{max}\}$ respectively

H_n : Weighted Harmonic mean download rate for the first n segments

Initialization:

if $B_{curr} \leq I$ // Fast Start

then

$l_{n+1} = r^{min}$

else

 if $(\frac{w_{n+1}}{H_n}) > B_{curr} - I$ then

$l_{n+1} =$

$\max\{r^i | r^i \in \mathbb{R}, \frac{w_{n+1}}{H_n} \leq B_{curr} - I, i \leq curr\};$

$\delta = 0;$

 else if $B_{curr} \leq B_\alpha$ // Additive Increase

 then

 if $\frac{w_{n+1}}{H_n} < B_{curr} - I$ then

$l_{n+1} = r^{curr} \uparrow$ // increase by one level

 else

$l_{n+1} = r^{curr};$

$\delta = 0;$

 else if $B_{curr} \leq B_\beta$ // Aggressive Switching

 then

$l_{n+1} =$

$\max\{r^i | r^i \in \mathbb{R}, \frac{w_{n+1}}{H_n} \leq B_{curr} - I, i \geq curr\};$

$\delta = 0;$

 else if $B_{curr} > B_\beta$ // Delayed Download

 then

$l_{n+1} =$

$\max\{r^i | r^i \in \mathbb{R}, \frac{w_{n+1}}{H_n} \leq B_{curr} - B_\alpha, i \geq curr\};$

$\delta = B_{curr} - B_\beta$

 else

$l_{n+1} = r^{curr};$

$\delta = 0$

Result:

l_{n+1} : the bitrate of the next segment to be downloaded

δ : The wait time before downloading the next segment

- **Additive increase** ($I < B_{curr} \leq B_\alpha$): Once the buffer occupancy goes beyond I , the algorithm enters the additive increase stage. In this stage, the bitrate is incremented in small (single) steps to avoid the buffer going back below I and thus, taking a conservative approach.
- **Aggressive switching** ($B_\alpha < B_{curr} \leq B_\beta$): The region between B_α and B_β is the most preferred buffer

occupancy. In this stage, based on the current network bandwidth and the buffer occupancy, the most suitable bitrate that is greater than or equal to the current bitrate is selected. This ensures a gradual and steady increase in the video quality.

- **Delayed Download** ($B_\beta < B_{curr} \leq B_{max}$): When the buffer occupancy increases beyond B_β , the most suitable bitrate for the current network bandwidth is selected; however, the request for the segment is sent only when the buffer occupancy falls to B_β . The delayed download limits the total number of segments in the video buffer, thus avoiding unnecessary downloads in case the user prematurely quits watching the video.

The rate adaptation algorithm, SARA, is presented in Algorithm. 1. SARA is invoked after downloading each segment to determine the representation to be selected for the next segment. The algorithm is initialized with the list of the available video bitrates (representations), $\mathbb{R}$ and the thresholds for the buffer I (default=2), B_α , B_β , B_{max} . These parameters remain constant during the entire playback.

SARA is invoked with the following input: the segment number of the most recent segment that was downloaded (say n ; 0 if none), the current bitrate (r^{curr} ; default is r^{min}), current buffer occupancy (B_{curr}), the sizes of $(n+1)^{th}$ segment across all the available representations ($w_{n+1}^{min}, \dots, w_{n+1}^m, \dots, w_{n+1}^{max}$), and the current weighted harmonic mean download rate.

At the start of the video session (or after an interruption due to buffer starvation), the number of segments in the buffer is below I and the playback begins in the *Fast Start* phase, during which the lowest bitrate is selected. Also, the delay factor δ is set to zero so that the next segment is downloaded immediately. Once the buffer occupancy increase beyond I , the adaptation enters the *Additive Increase* stage.

The time required to download the next segment that corresponds to the current bitrate is given by (w_{n+1}^{curr}/H_n) . At any time during the playback, it is not feasible to download the next segment of the current bitrate before the buffer goes below I , if the time taken to download the next segment is greater than $B_{curr} - I$; hence, the best possible bitrate that can be downloaded in the duration $B_{curr} - I$ is selected based on the next segment size and the current weighted harmonic mean download rate (H_n). SARA tries to maintain the buffer occupancy above I ; else, it switches to the lowest bitrate (r^{min}).

In the additive increase stage, the algorithm starts the adaptation process during which the video bitrate is gradually increased. The increase in the quality is done in single steps; $r^{curr} \uparrow$ indicates the next level of the bitrate. By using single step increments we ensure gradual quality changes, thus avoiding the unpleasant effect of rapid bitrate switches. However, the step size is a configurable parameter that can be based on the desired aggressiveness. When the video session is in the additive increase stage, i.e., the current buffer occupancy (B_{curr}) is below B_α . If the amount of time needed to download the next segment of the bitrate, which is one more than the current bitrate i.e., $r^{curr} \uparrow$ is less than $B_{curr} - I$, then the next level is chosen. Otherwise, the current bitrate is

maintained.

While $B_{curr} < B_\alpha$, SARA takes a conservative approach of giving a higher priority to minimize the start time and interruptions over the video quality. Once B_{curr} goes beyond B_α , it starts to be more optimistic. While $B_\alpha < B_{curr} < B_\beta$ the highest bitrate whose segment could be downloaded before the current buffer goes below I ($w_{n+1}^i/H_n \leq B_{curr} - I$) is selected. By utilizing the segment sizes in the decision making process, we ensure that even when the segment sizes fluctuate, the download time of the segment can be predicted with better accuracy. In the *aggressive switching* stage, the next segment request is sent immediately. The algorithm selects the highest possible video rate, while making sure that the buffer is maintained at a steady level.

When the network conditions are favorable and the buffer occupancy increases beyond B_β , the algorithm selects the best possible bitrate similar to the aggressive switching stage, but instead of sending the request for the next segment immediately, it waits until the buffer occupancy falls to B_β . This limits an unnecessary data fetch in case the user decides to prematurely quit watching the video. The algorithm returns the bitrate for the next segment and the waiting time to issue the request for the next segment.

IV. EVALUATION AND RESULTS

The DASH video players used by the popular service providers such as Netflix, YouTube, and Hulu are proprietary and hence, difficult to modify. To evaluate the performance of SARA we compared it with a basic rate adaptation algorithm. We first developed an emulated DASH client written in Python. Using this player we were able to emulate a DASH video session that can be easily configured to work with any rate adaptation algorithm. The code for the emulated DASH client is available from our Github project page [13].

Our emulated DASH client has four modules: (1) the *MPD parser module* that downloads and parses the MPD to obtain the segment information (available bitrates, segment URLs, and segment sizes); (2) the *segment download module* that downloads the segment from the DASH server; (3) the *video buffer module* that emulates a video player's buffer; and (4) the *rate adaptation module* that is used to determine the representation for the next segment to be downloaded. The *rate adaptation module* currently supports the adaptations as in the basic and SARA schemes, but can be easily modified to use any other adaptation. The player is initialized with the URL of the MPD file. The *MPD parser module* downloads and parses the MPD file to retrieve the list of all representations and segment URLs. Next, the *rate adaptation module* is invoked to determine the suitable representation for the segment. The selected segment is then downloaded by the *segment download module* and placed into the video buffer. Once the number of segments in the video buffer increases beyond the initial threshold, the emulated video player starts reading the buffer one segment at a time.

A client running the emulated video player and an Apache HTTP web server were deployed on the GENI networking testbed [14]. Both machines were running Ubuntu 12.04 OS.

GENI is a geographically distributed networking testbed for future internet applications. The bandwidth between the server and client was controlled using an HTTP outgoing traffic shaper, tc [15], at the server.

The basic rate adaptation scheme used in this work uses the average segment-download rate and the current buffer occupancy. It starts by downloading the segment with the lowest bitrate. While downloading the first two segments (same as I in SARA) with the lowest bitrate, the average download rate of the segments is measured. When the buffer occupancy is greater than two segments, the bitrate is increased from the lowest bitrate. Based on the measured average throughput and the current buffer occupancy, it selects the bitrate for the next segment, which is one less than the the bitrate closest to the available bandwidth. The buffer size for the basic adaptation was set to a maximum of 10 segments (40 seconds) and uses an optimistic switch-up and switch-down bitrate switching.

For evaluation, we used the popular open-source movie *Buck Bunny* [7], [16] that has been used in other studies. The open source version of this video is available in 20 different representations. The total length of the video is around 596 seconds, and each representation is split into 150 segments, each with a 4 second playback duration. The Apache web server hosts two different MPD files: the basic MPD file (for basic adaptation) and the extended MPD file (for SARA).

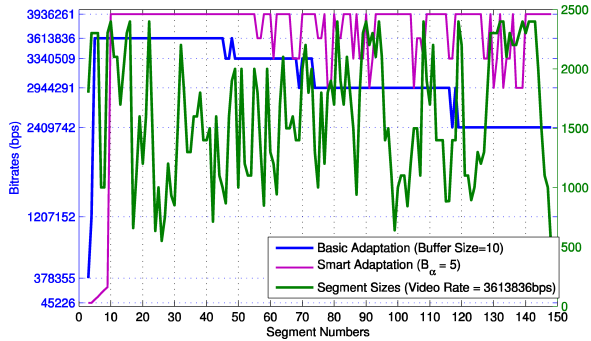


Fig. 3. Effect of Segment Sizes on Basic and SARA (Bandwidth = 1 Mbps)

First, we studied the effect of the variation in the segment sizes on the two adaptation schemes. We did this by limiting the network bandwidth between the client and the server to 1 Mbps. The buffer size of the basic adaptation algorithm was configured to be 10 segments and SARA was configured with $B_\alpha = 5$, $B_\beta = 10$, and $B_{\max} = 12$. The video was played using both ABR algorithms, the experiment being repeated 10 times. The results reported are based on the statistical mode of the bitrate values measured, since video qualities are at discrete values. In Fig. 3, we show the variation in the video bitrate selected by each ABR along with the segment sizes (in KB). Even with a steady bandwidth we noticed that the variation in the segment sizes affected the segment download times. Although the sudden changes in the segment sizes affected the bitrate selected by both the schemes, the basic algorithm failed to anticipate the decrease in the segment sizes, forcing it to pick a lower bitrate. However, since SARA had the segment information it was able to predict the segment download times

and thus, maintained higher bitrates for longer durations.

Next, we studied the bitrate adaptation for both the algorithms with the network bandwidth fixed at 4 Mbps and 8 Mbps. The results are shown in Fig. 4 and Fig. 5, respectively. In both the cases, we observed that due to the *Additive Increase* stage of SARA, it takes a longer time to reach the highest bitrate compared to the basic ABR algorithm. Particularly, with $B_\alpha = 10$, it takes about 20 seconds to reach the highest bitrate. However, once SARA reaches the highest bitrate, it sustains the higher bitrate for longer durations. Even with $B_\alpha = 5$, we see that the bitrate picked by SARA was always better than (or at least as good as) the basic adaptation. With $B_\alpha = 10$, SARA converges to the highest bitrate steadily, but at a slower rate than the basic scheme. Subsequently, there are no bitrate switches when the segment sizes vary. Having a larger buffer gives a cushion to SARA and avoids responding to sudden changes in the segment download rates. The use of a buffer size of 10 segments translates to a video playback of 40 seconds, which seems reasonable considering that a user may watch a movie that may last 30 minutes or longer as is the case with most videos on Netflix.

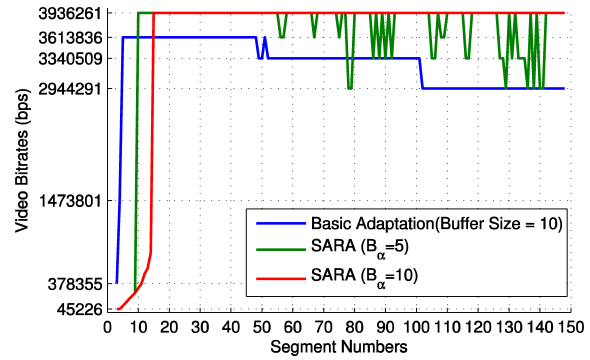


Fig. 4. Bitrate Variation when Bandwidth = 4 Mbps

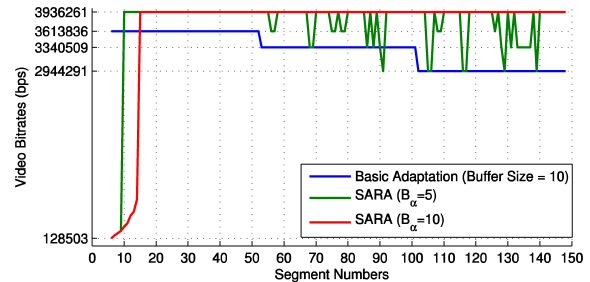


Fig. 5. Bitrate Variation when Bandwidth = 8 Mbps

To understand the performance of the SARA algorithm better, we define the relative video quality measure Q_{ratio} as the ratio of the total quality of the video played by SARA to the basic adaptation algorithm. Let $f_{SARA}(x)$ and $f_{basic}(x)$ denote the bitrate selected for a segment x by SARA and the basic ABR algorithms, respectively. The Q_{ratio} is defined by

$$Q_{ratio} = \frac{\int f_{SARA}(x) dx}{\int f_{basic}(x) dx} \quad (2)$$

TABLE I
Q RATIO

Bandwidth	Video Quality Measure (Q_{ratio})	
	SARA ($B_\alpha = 5$)	SARA ($B_\alpha = 10$)
8 Mbps	1.13	1.15
6 Mbps	1.15	1.16
4 Mbps	1.16	1.16
1 Mbps	1.22	1.23

The Q_{ratio} obtained for the SARA algorithm with different buffer thresholds, under varying network bandwidths are reported in Table I. For the second case, the thresholds were set to $B_\alpha = 10$, $B_\beta = 15$, and $B_{max} = 17$. As shown in earlier results, increasing B_α to 10 leads to a steady improvement in the bitrate of the video played. For 8 Mbps bandwidth, both the cases of SARA showed a 13% and 15% increase in quality over the basic scheme. We found that for lower bandwidths, the performance of SARA was even better, with SARA ($B_\alpha = 5$) having 22% and SARA ($B_\alpha = 10$) having 23% better quality, respectively. Overall, the quality of the video played by SARA was at least 13% higher.

Apart from the quality of the video, the number of bitrate switching events also affects the QoE [17]. Even though, SARA with $B_\alpha = 5$ improved the playback quality (as observed from Q_{ratio}), the number of bitrate switching events is higher than the basic algorithm. However, with $B_\alpha = 10$ we observed that the video playback was much smoother. We also report the total number of bitrate switching events in Table II. We see that the number of bitrate switches increases for the basic adaptation as the download rate decreases. For SARA with $B_\alpha = 10$, the number of switching events is constant, at 11. These 11 bitrate switching events correspond to the segments downloaded during the *FastStart and AdditiveIncrease* stages. Once the threshold of 10 is reached, SARA exits the additive increase stage and sustains the highest bitrate. Even though the bitrate switches are comparatively higher than the basic adaptation for higher bandwidths (8, 6 and 4 Mbps), it must be noted that these are small upward increments. On the other hand, the basic adaptation scheme has downward switches. The polarity of the switching also affects the QoE. It was reported in [18] that the users are more critical towards downward switches and less affected by upward switching, which indicates that the increase in the number of upward switches to avoid subsequent downward switches can lead to a better QoE. On the other hand, a recent study [19] has shown that for higher segment sizes (4 and 10 sec.), frequent bitrate switching is not perceived significantly worse than videos with less bitrate switches.

V. SUMMARY AND FUTURE WORK

In this paper, we presented a Segment Aware Rate adaptation (SARA) algorithm for DASH video players. This adaptation scheme enhances the MPD file to contain the segment sizes and facilitate SARA to account for the effect of variable segment sizes on the measured bandwidth accordingly. We compared our algorithm with a basic adaptation scheme that uses only the segment download rate, and demonstrated that under low network bandwidth, our scheme has better advantages. The improvement in performance was measured using

TABLE II
BITRATE SWITCHING EVENTS

Bandwidth	Number of bitrate switching events		
	Basic	SARA ($B_\alpha = 5$)	($B_\alpha = 10$)
8 Mbps	4	38	11
6 Mbps	4	44	11
4 Mbps	6	40	11
1 Mbps	11	41	11

two metrics: the relative quality measure (Q_{ratio}) and the number of bitrate switching events. As a part of our future work, we plan to implement and evaluate SARA on a DASH video player that works with an existing video service. In our initial evaluation, we kept the buffer thresholds constant. We also plan to update SARA to dynamically adjust these thresholds and pick optimal values for the current session to limit unnecessary downloads of segments.

REFERENCES

- [1] I. Sodagar, "The MPEG-DASH standard for multimedia streaming over the internet," *MultiMedia, IEEE*, vol. 18, no. 4, pp. 62–67, April 2011.
- [2] Sandvine, "Global Internet Phenomena Report: 1H 2014." [Online]. Available: <http://www.sandvine.com/downloads/general/global-internet-phenomena/2014/1h-2014-global-internet-phenomena-report.pdf>
- [3] C. Liu, I. Bouazizi, and M. Gabbouj, "Rate adaptation for adaptive HTTP streaming," in *Proc. of the 2nd ACM conf. on Multimedia systems*, 2011, pp. 169–174.
- [4] R. K. Mok, X. Luo, E. W. Chan, and R. K. Chang, "QDASH: a QoE-aware DASH system," in *Proc. of 3rd Multimedia Systems*, 2012, pp. 11–22.
- [5] J. Jiang, V. Sekar, and H. Zhang, "Improving fairness, efficiency, and stability in HTTP-based adaptive video streaming with FESTIVE," in *Proc. of the 8th Emerging networking experiments and technologies*, 2012, pp. 97–108.
- [6] T.-Y. Huang, R. Johari, N. McKeown, M. Trunnell, and M. Watson, "A buffer-based approach to rate adaptation: Evidence from a large video streaming service," in *Proc. of the ACM SIGCOMM*, 2014, pp. 187–198.
- [7] "The Big Buck Bunny Movie," [online] <http://www.bigbuckbunny.org/>
- [8] P. Juluri, V. Tamarapalli, and D. Medhi, "Look-Ahead rate adaptation algorithm for DASH under varying network environments," in *11th DRCN 2015 (Poster paper)*, 2015.
- [9] S. Akshabi, A. C. Begen, and C. Dovrolis, "An experimental evaluation of rate-adaptation algorithms in adaptive streaming over HTTP," in *Proc. of the 2nd ACM conf. on Multimedia systems*, 2011, pp. 157–168.
- [10] K. Miller, E. Quacchio, G. Gennari, and A. Wolisz, "Adaptation algorithm for adaptive streaming over HTTP," in *Packet Video Workshop (PV), 2012 19th International*, 2012, pp. 173–178.
- [11] J. Heidemann, K. Obraczka, and J. Touch, "Modeling the performance of HTTP over several transport protocols," *Networking, IEEE/ACM Trans. on*, vol. 5, no. 5, pp. 616–630, 1997.
- [12] S. Krishnan and R. Sitaraman, "Video stream quality impacts viewer behavior: Inferring causality using quasi-experimental designs," *Networking, IEEE/ACM Trans. on*, vol. 21, no. 6, pp. 2001–2014, Dec 2013.
- [13] "AStream: A rate adaptation model for DASH." [Online]. Available: <https://github.com/pari685/AStream>
- [14] M. Berman, J. S. Chase, L. Landweber, A. Nakao, M. Ott, D. Raychaudhuri, R. Ricci, and I. Seskar, "GENI: a federated testbed for innovative network experiments," *Computer Networks*, vol. 61, pp. 5–23, 2014.
- [15] A. N. Kuznetso, "Traffic control settings." [Online]. Available: <http://manpages.ubuntu.com/manpages/precise/man8/tc.8.html>
- [16] S. Lederer, C. Müller, and C. Timmerer, "Dynamic adaptive streaming over HTTP dataset," in *Proc. of the 3rd Multimedia Systems Conf. ACM*, 2012, pp. 89–94.
- [17] L. Yitong, S. Yun, M. Yinian, L. Jing, L. Qi, and Y. Dacheng, "A study on quality of experience for adaptive streaming service," in *IEEE International Conf. of ICC*, 2013, pp. 682–686.
- [18] N. Cranley, P. Perry, and L. Murphy, "User perception of adapting video quality," *International Journal of Human-Computer Studies*, vol. 64, no. 8, pp. 637–647, 2006.
- [19] S. Egger, B. Gardlo, M. Seufert, and R. Schatz, "The impact of adaptation strategies on perceived quality of HTTP adaptive streaming," in *Proc. of the Workshop on Design, Quality and Deployment of Adaptive Video Streaming*. ACM, 2014, pp. 31–36.